

Recap

TCP congestion control:

Slow start, Congestion Avoidance (AIMD), Fast recovery

* Signals considered by TCP for congestion control

- 1) Timeout
- 2) 3 dup Acks

* Slow start

1. set $cwnd = 1MSS$
2. send 'cwnd' bytes during transmission round
3. If all Acks arrive on time (ie, No timeout during round)
 - a. If $cwnd < ssthresh$; double cwnd (max val: ssthresh.)
 - b. else Enter congestion avoidance phase (AIMD)
4. If timeout occurs for some segment 's'
 - a. retransmit S
 - b. set $ssthresh = \frac{cwnd}{2}$

* Congestion avoidance (AIMD)

1. For any given transmission round; if all Acks arrive on time; $cwnd = cwnd + 1MSS$
2. If timer expires; $ssthresh = cwnd/2$; enter Slow start.

* Fast recovery (If 3 dup Acks recd. TCP Tahoe RT, ssthresh & " TCP Reno retransmits S & does fall-)

- For TCP Reno
1. $ssthresh = cwnd/2$
 2. $cwnd = ssthresh + 3MSS$ for 3 dup Acks cwnd inflation
 3. For every incoming dup Ack thereafter, $cwnd = cwnd + 1MSS$ ie, allowed to send extra data
 4. When new Ack arrives; $cwnd = ssthresh$; switch to congestion avoidance phase

* Fast recovery * TCP Tahoe $\rightarrow$ x fast recovery

o TCP Reno

- Ack x

- Ack x

- Ack x

- Ack x

dup Ack count = 0

1 ✓

2 ✓

3 $\Rightarrow$

3 dup Ack

$\rightarrow ssthresh = cwnd / 2 + 1 + 1 + 1$ } retransmitted segment with dup ack seg #
 $\rightarrow cwnd = ssthresh + 3MSS$

more

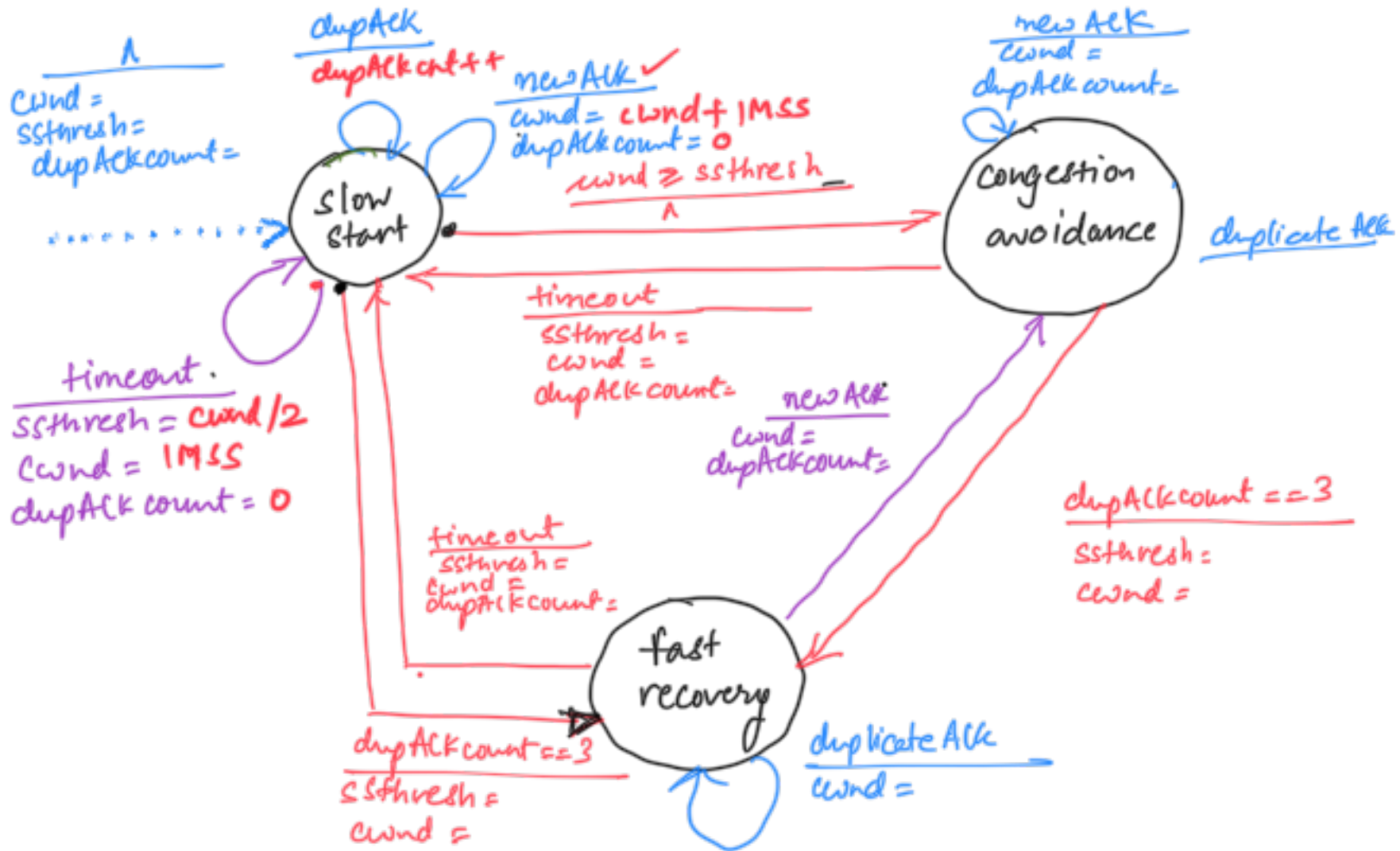
dup Ack $\Rightarrow$ each dup Ack $\Rightarrow cwnd = cwnd + 1MSS$

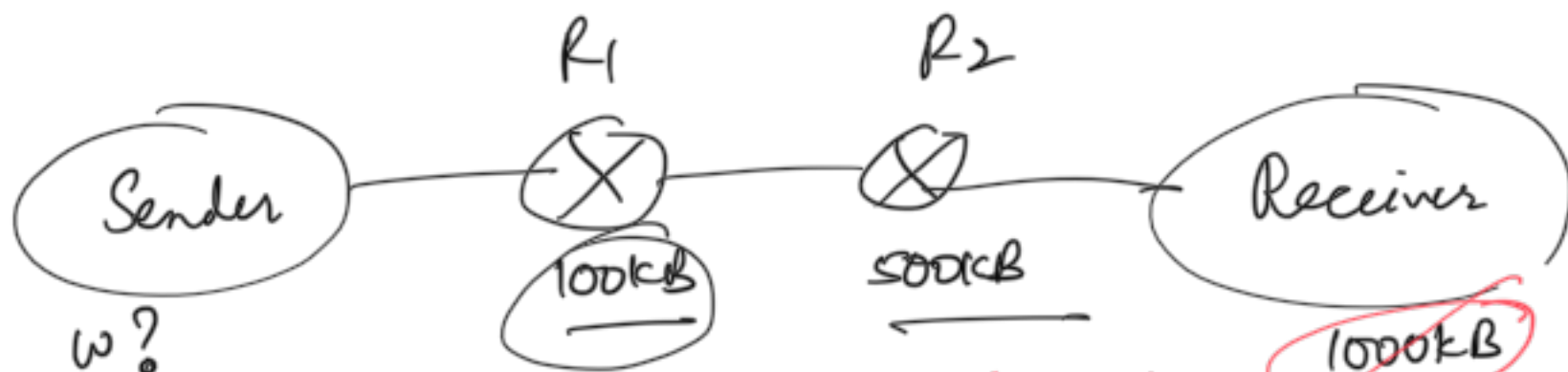
new Ack

(Ack that fills the gap)

$cwnd = ssthresh$

TCP Congestion Control





- Receiver buffer size ($rwnd$) shared as part of TCP header
 - congestion window size ($cwnd$) estimate using congestion control algo
- Sender window size = $\min(rwnd, cwnd)$

